Study Report: Python - 2 Books in 1 (Python for Beginners + Python Programming)

Written by Gagan Pasupuleti

Family: Combo Learning Paths

Level: Mixed

Chapters: 7

Source: Python - 2 books in 1 - Python For Beginners + Python Programming.pdf

Summary

This report covers a two-in-one bundle that pairs a gentle beginner book with a more complete programming guide. It builds from Python foundations (setup, variables, data types, control flow) into working with collections and files, then moves toward data handling, simple analysis, and an introduction to machine learning ideas, ending with combined practice. It gives learners both a solid start and a preview of where Python leads.

Chapters

Chapter 1: Python Foundations

Chapter focus: Python Foundations

Python is a general-purpose programming language known for readable syntax and wide use in web development, automation, data analysis, and machine learning. Unlike many languages that use braces and semicolons, Python uses indentation to show structure, which helps beginners see how code is organized. You write instructions in a .py file or run them line by line in the interactive shell. Each instruction tells the computer to do something: store a value, print text, make a decision, or repeat a task.

Code Reference:

```
print("Hello, Python!")
print(2 + 3)
```

What it shows: The first line prints text. The second shows Python can also calculate numbers.

Topic guide

What it actually is

Python is a high-level, interpreted language: you write human-readable code and the Python interpreter runs it without a separate compile step. It comes with a large standard library and thousands of third-party packages.

When to use it
Use Python when you want to build something quickly, automate repetitive tasks, analyze data, create a web backend, or learn programming for the first time.

Where to use it
Used at Google, Netflix, NASA, and in schools worldwide. Common in data science teams, DevOps automation, scripting, and beginner coding courses.

Real use example
A small business owner writes a 10-line Python script that renames 200 invoice PDFs by date every month, saving an hour of manual work.

Chapter 2: Data Types and Control Flow

Chapter focus: Data Types and Control Flow

A variable is a name that points to a value stored in memory. You create one with the assignment operator (=): the name goes on the left, the value on the right. Python figures out the type automatically - you do not declare int or str first. Common types are integers (whole numbers), floats (decimals), strings (text in quotes), and booleans (True/False). You can change a variable later by assigning a new value. Clear names like user_age or total_price make code easier to read than single letters.

Code Reference:

```
count = 5          # integer
label = "box"      # string
price = 2.5        # float
is_open = True     # boolean
print(type(count))  # <class 'int'>
```

What it shows: Each variable stores one value. type() tells you what kind of data it holds.

Topic guide

What it actually is
Variables are labeled containers for data. The label (name) lets you reuse and update values throughout a program without repeating the actual number or text.

When to use it
Use variables whenever a value might change, will be reused later, or needs a meaningful name - scores, user names, prices, flags, counters.

Where to use it
Every Python program: games (player score), web apps (username), data scripts (file path), calculators (operands), and loops (counters).

Real use example
An online shop stores item_price = 499, quantity = 2, and computes total = item_price * quantity so the checkout page shows Rs 998 without hard-coding numbers.

Chapter 3: Working with Data Collections

Chapter focus: Working with Data Collections

A list stores multiple items in order inside square brackets. Items can be added with .append(), removed with .remove() or .pop(), and accessed by index (0-based). Lists are mutable, so you can sort in place, insert in the middle, or change one item. Negative indexing (-1) gets the last item. Lists are the most common way to store a collection you will change - scores, names, lines from a file, or results from a loop.

Code Reference:

```
scores = [88, 92, 75]
scores.append(90)
scores.sort()
print(scores[0], scores[-1])  # 75 92
```

What it shows: append adds an item; sort orders the list; [0] is first, [-1] is last.

Topic guide

What it actually is
A list is a mutable, ordered collection. Order matters: ['a','b'] is different from ['b','a'].

When to use it
Use a list when you have many values of the same kind and need to add, remove, sort, or loop through them.

Where to use it
Shopping cart items, test scores, lines read from a file, todo tasks, or collecting results in a loop.

Real use example
A teacher stores exam marks in marks = [78, 85, 90], appends a late submission, sorts the list, and prints the lowest and highest for a class summary.

Chapter 4: Functions and Files

Chapter focus: Functions and Files

Keep functions small and named after what they do. Document tricky ones with a one-line docstring. Return early when possible to avoid deep nesting. Use pathlib.Path for modern path handling. Check Path('data.csv').exists() before reading. Encoding='utf-8' avoids character errors on Windows.

Code Reference:

```
def area(width, height):
    return width * height

def print_area(w, h):
    print(area(w, h))
```

```
print_area(4, 5)  # 20
```

*What it shows: area computes and returns; print_area reuses area instead of duplicating w * h.*

Topic guide

What it actually is
A function is a named, reusable mini-program inside your program. You call it by name with arguments; it may return a result.

When to use it
Use a function when the same logic appears twice, when a task has a clear name, or when you want to test one piece separately.

Where to use it
Calculations, formatting output, validating input, API handlers, and splitting large scripts into readable pieces.

Real use example
A backup script copies every .docx from ~/Documents to ~/Backup using shutil and Path.glob('*.docx').

Chapter 5: Introduction to Data Analysis

Chapter focus: Introduction to Data Analysis

Python is a general-purpose programming language known for readable syntax and wide use in web development, automation, data analysis, and machine learning. Unlike many languages that use braces and semicolons, Python uses indentation to show structure, which helps beginners see how code is organized. You write instructions in a .py file or run them line by line in the interactive shell. Each instruction tells the computer to do something: store a value, print text, make a decision, or repeat a task.

Code Reference:

```
print("Hello, Python!")
print(2 + 3)
```

What it shows: The first line prints text. The second shows Python can also calculate numbers.

Topic guide

What it actually is
Python is a high-level, interpreted language: you write human-readable code and the Python interpreter runs it without a separate compile step. It comes with a large standard library and thousands of third-party packages.

When to use it
Use Python when you want to build something quickly, automate repetitive tasks, analyze data, create a web backend, or learn programming for the first time.

Where to use it

Used at Google, Netflix, NASA, and in schools worldwide. Common in data science teams, DevOps automation, scripting, and beginner coding courses.

Real use example
A small business owner writes a 10-line Python script that renames 200 invoice PDFs by date every month, saving an hour of manual work.

Chapter 6: Introduction to Machine Learning Ideas

Chapter focus: Introduction to Machine Learning Ideas

Always split data into train and test sets. Never evaluate only on data the model already saw - that inflates scores misleadingly.

Code Reference:

```
print("Hello, Python!")
print(2 + 3)
```

What it shows: The first line prints text. The second shows Python can also calculate numbers.

Topic guide

What it actually is
Python is a high-level, interpreted language: you write human-readable code and the Python interpreter runs it without a separate compile step. It comes with a large standard library and thousands of third-party packages.

When to use it
Use Python when you want to build something quickly, automate repetitive tasks, analyze data, create a web backend, or learn programming for the first time.

Where to use it
Used at Google, Netflix, NASA, and in schools worldwide. Common in data science teams, DevOps automation, scripting, and beginner coding courses.

Real use example
A fruit classifier trains on 800 labeled photos, tests on 200 held-out photos, and reports 88% accuracy on the test set only.

Chapter 7: Putting Skills Together

Chapter focus: Putting Skills Together

A project combines many skills into one working program: input, logic, data structures, maybe files or libraries. Plan small - one clear goal, a few features, then test. Break the project into functions or steps. Debugging is normal. Finishing a project teaches more than reading ten chapters because you see how pieces connect.

Code Reference:

```
# Mini project: guess the number
```

```
import random
secret = random.randint(1, 10)
for _ in range(3):
    g = int(input("Guess 1-10: "))
    if g == secret:
        print("Correct!"); break
else:
    print("Out of tries. Answer:", secret)
```

What it shows: Combines random, input, loop, conditionals, and break in one playable game.

Topic guide

What it actually is
A project is a complete program that solves a small real problem, not just a single concept demo.

When to use it
After learning basics - consolidate skills, portfolio pieces, homework capstones.

Where to use it
Courses, hackathons, personal GitHub, job interviews (show working code).

Real use example
A student builds a contact book CLI that saves names and phones to a JSON file - uses dicts, files, functions, and loops in one app.